UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

DATE:

13/05/2026

CLASS:

AWD - 30716

HOMEWORK:

Walkthrough Review

List of founded errors, missing elements and inconsistencies

- **Andrés Cárdenas**
  - **Product Backlog Errors**
    1. **Overly Technical Language in User Stories**: User Stories like **US-03** and **US-05** include technical implementation details such as "API", "GET", "PUT", and "DELETE". According to Agile standards, User Stories should describe the "what" and "why" from a user's perspective, leaving the "how" (technical details) for the Acceptance Criteria or technical tasks.
    2. **Ambiguous Field Definitions**: In **PB-04**, the story description mentions capturing the "reason for consultation," but this field is omitted from the specific list in the Acceptance Criteria. This leads to a lack of clarity for the developer on whether that specific data point needs validation or even storage.
  - **Product Backlog Gaps**
    3. **Lack of Patient-Payment Integration**: **PB-09** requires capturing a "Patient ID number" for payments , but the backlog lacks a search or selection feature (lookup). This forces the receptionist to manually type a 10-digit ID every time, increasing the risk of human error and orphaned records.
    4. **Incomplete Legal Representative Data**: While **PB-08** and **RF-03** mandate a legal representative for minors , the backlog never specifies which data points are required for this person (e.g., Name, ID, or Phone). Without these details, the registration is legally insufficient for a pediatric clinic.
    5. **Absence of "Soft Delete" for Records**: **PB-07** and **US-03** allow for the total deletion of patient records from the database. In a medical context, completely removing data is a risk; the system is missing a "Deactivation" or "Archive" feature to preserve medical history for legal purposes.
  - **Product Backlog Inconsistencies**
    6. **Contradictory Acceptance Criteria (PB-04)**: The description for **PB-04** lists specific fields to capture, but the "Acceptance Criteria" column for the same item provides a slightly different set of requirements, creating confusion about the final scope of the registration form.
    7. **Sprint Status vs. Item State**: In the Sprint Planning table, **Sprint 3** is marked as "Filled" (Completed). However, **PB-12** (Visual indicator of payment status), which is part of that same Sprint, is still marked as "Earring" (Pending).
  - **Use Case Diagram Errors**
    8. **Typo in Actor Name**: The actor representing the receptionist is explicitly misspelled as **"Rcepcionist"** but the correct name should be **"Receptionist"**.

➢ **Use Case Diagram Gaps**

9. **Missing "Visitor" Actor**: The initial backlog establishes in US-08 that a "Clinic visitor" needs to view available treatments without logging in. This external actor and their corresponding use case are entirely absent from the diagram.

➢ **Class Diagram Errors**

10. **Missing Dependency Lines:** The Dentist and Receptionist classes contain methods that use Patient and Payment objects as parameters (e.g., registerPatient(p: Patient)). However, there are no UML dependency arrows (dashed lines with open arrowheads) connecting these actor classes to the entity classes they manipulate, which is required in proper UML syntax to show how classes interact.

➢ **Class Diagram Gaps**

11. **Missing Age Calculation Logic:** The Patient class has a birthday attribute and a generic validateData() method. However, since the backlog strictly requires a legal representative only if the patient is under 18, the class is missing a specific behavioral method (e.g., calculateAge()) to trigger that validation dynamically.

➢ **Class Diagrama Inconsistencies**

12. **Security Implementation Discrepancy:** The Product Backlog explicitly states in PB-20 that user passwords must be encrypted using bcrypt hashing for security. The User parent class simply lists -password: string and a basic login() method, failing to include any methods for password hashing, verification, or token generation that would reflect this security requirement in the backend architecture.
13. **Method Naming vs. Parameter Arity**: In the Receptionist class, there is a method named registerPayments(p: Payment) (plural). However, it only accepts a single Payment object as a parameter. This contradicts both its own signature and the backlog (PB-09), which describes registering an individual payment per transaction.

➢ **Code Test Cases**

14. **Submit patient registration with a 9-digit ID number**
    The form prevents submission and displays an error indicating the format is not correct.
15. **Register a minor patient leaving the legal representative field empty**
    The system dynamically validates the age and blocks the submission, highlighting the Legal Representative field as required.
16. **Submit patient registration leaving the undocumented 'Gender' field unselected**
    The system prompts the user to select an option from the dropdown before allowing the save action.
17. **Register a patient selecting a future date of birth**
    The date picker disables future dates.

18. **Register a new patient using an already existing ID number**
    The system creates a duplicate row with the exact same ID number.
19. **Filter patient list typing a partial name in the search bar**
    The table dynamically filters to show only "Alejandro Arroyo" in real-time as you type.
20. **Click edit button and modify patient phone number**
    The UI changes to edit mode, saves the PUT request, and the new phone number is visible.
21. **Click delete button on patient record to verify UI removal**
    The record disappears from the table immediately after confirmation.
22. **Submit a payment typing an unregistered patient ID number**
    The system accepts the payment, creating an orphaned financial record not linked to any real patient.
23. **Record a payment inputting a negative value in the amount field**
    The form validation blocks the negative input and prompts for a value greater than 0.
24. **Submit payment form leaving the amount field empty**
    The form cannot be submitted, and the browser highlights the amount field as required.
25. **Submit payment selecting 'Abono' to verify 'Parcial' status assignment**
    The payment is saved and automatically appears in the History table with the yellow "Parcial" status badge.
26. **Submit payment selecting 'Final' to verify 'Completed' status assignment**
    The status remains yellow "Parcial" or requires manual updating.
27. **Record a payment selecting a future date in the calendar**
    The system saves the payment with a future date, creating a temporal error in the clinic's accounting.
28. **Click edit button on payment history and modify payment amount**
    The system successfully updates the amount via API and reflects the $40.00 immediately in the table.
29. **Click delete button on payment record to check UI and database removal**
    The DELETE request succeeds and the payment row is instantly removed from the UI.
30. **Submit a payment leaving the dropdown menus in their default unselected state**
    The frontend validation blocks the submission and displays an error message requiring the user to select a valid option from the dropdown lists.

❖ **Kerly Chuqui**

➢ **Architecture and Databases**

1. **Database Engine Contradiction:** The Product Backlog explicitly specifies the use of "MongoDB Atlas" for all collections. However, the dbCredentials.php file contains credentials to connect to a PostgreSQL database hosted on Supabase using Eloquent ORM.

2. **Unintentional Split Architecture:** api_patients.php connects directly to MongoDB, while supply-controller.php invokes dbCredentials.php (PostgreSQL). Data is fragmented across two completely different database engines, violating the Backlog requirements.

3. **Missing Actor Classes:** The Class Diagram defines the User, Dentist, Administrator, and Receptionist classes with methods like registerPatient(). In the code, object-oriented programming is incomplete: these classes do not exist, and all logic relies on procedural controllers.

4. **Critical Security (Lack of Patient Authentication):** The RNF-01 requirement mandates session control. However, api_patients.php does not execute session_start() or validate roles. Any anonymous user can send HTTP requests and delete/create patients directly from tools like Postman.

5. **Exposed Credentials (Hardcoding):** Despite security requirements, api_patients.php and dbCredentials.php contain the production database passwords exposed in plain text.

➢ **Patient Module**

6. **Use Case Permission Violation:** The Backlog (PB-02) clearly states that "The Dentist only has access to Patients" and the Receptionist only to Payments. However, the Use Case Diagram connects the Receptionist to the actions "Register Patient," "View Patient List," and "Search Patient."

7. **Incorrect Actor for Search:** Related to the above, Backlog PB-18 requires the Dentist to search for patients, contradicting the Use Case Diagram, which assigns this task to the Receptionist.

8. **Missing Association in Class Diagram:** Although the Use Case Diagram allows the Receptionist to manage patients, the Class Diagram does not draw any relationship between the Receptionist class and the Patient class.

9. **Missing Patient ID Restriction (Classes):** Backlog (PB-04) strictly requires a 10-digit ID number, but the Class Diagram defines patientID as a simple string, without modeling this restriction.

10. **ID Validation Error:** The code in Patient.php (validateData()) checks the length using strlen($this->patientID) !== 10, but never verifies that the numbers are indeed numbers. Text like "ABCDEFGHIJ" would be saved successfully.

11. **Incomplete Data Validation:** PB-04 requires capturing "phone number" and "reason for consultation." While the Patient.php constructor maps these fields, validateData() does not check if they are empty. A patient can be created without a phone number or reason for consultation.

12. **Data Type Inconsistency (Birthday):** The Class Diagram indicates that birthday is of type DateTime. In the Patient.php code, this attribute is treated and stored as a normal string.

13. **Failure in Legal Representative Validation:** The Backlog (PB-08) states that a legal representative is mandatory for minors. The Patient class does not encapsulate this rule in its validation. This rule is loose and disorganized within the api_patients.php controller.

14. **Risk of Crashing Due to Age:** In api_patients.php, age is calculated by instantiating new DateTime($patient->birthday). If the date is null or in a corrupted format from the frontend, PHP will throw a fatal exception ("Uncaught Exception"), breaking the API instead of displaying a validation error.

15. **REST API Architecture Violation (POST):** According to requirement RF-09, RESTful APIs must be created. However, when saving a patient (POST) in api_patients.php, the code redirects the view using header('Location: ...') instead of returning a JSON object with the HTTP 201 Created status code.

16. **ID Key Conflicts in Edit (PUT):** PB-06 indicates that the patient will be updated via PUT. The code receives the request body and looks for a key ['id'] to update the native MongoDB ID ['_id']. In standard APIs, the ID should be included in the URL (e.g., /api_patients.php/123), and this mix will cause conflicts when consuming the API with Vue or JavaScript.

➢ **Supply Module (Inventory)**

17. **Typographical Errors in UML (Classes):** The class diagram indicates the attributes unitCoast, orderData, and expirationData. The backend code (correctly) calls them unitCost, orderDate, and expirationDate.

18. **State Discrepancy (Backlog vs. UML):** The Backlog (PB-14) requires the states to be 'Valid', 'Due', and 'Expired'. The Class Diagram requires the use of the SupplyStatus enumeration with the values 'Current', 'NextExpiration', and 'Expired'.

19. **State Discrepancy (Backlog vs. Code):** The code ignores what is stipulated in the Backlog. The calculateStatus() function in Supply.php assigns 'Current' and 'NextExpiration', 'Expired', and also invents a fourth state: 'Pending'.
20. **Quantity Validation Error:** PB-13 requires a minimum quantity of 1. The `isValidQuantity()` function checks if quantity > 0. Since it doesn't verify if the value is of type `int`, entering a floating-point number like 0.5 would be considered valid.

21. **Phantom Method:** The Class Diagram requires the existence of the `updateStock(qty: int): void` method in the `Supply` class. This method was not programmed in the `Supply.php` file.

22. **Incorrect Return Type:** The Class Diagram requires `calculateStatus()` to return the `SupplyStatus` enumerator. In the developed code, it simply returns a free-text string.

23. **Dead Code in Date Calculation:** `calculateStatus()` contains the fragment `if (empty($this->expirationDate)) { return 'Pending'; }`. This is inaccessible code ("dead code"), as the `areDatesValid()` method explicitly prohibits saving if the expiration date is empty.

24. **Inaccurate Expiration Calculation:** PB-15 alerts if there are <= 30 days remaining. The code uses `$interval->days <= 30 && $interval->invert == 0`. Because it forces the time to 00:00:00, if a supply expires today, the time behavior may unduly evaluate whether it is "Expired" or "NextExpiration".

25. **Fake REST API in Supplies:** The `supply-controller.php` controller is NOT a REST API. Everything is processed by sending the HTTP POST method and disguising the action in a `$_POST['action']` variable with values like 'create' or 'delete', instead of using true HTTP verbs (PUT/DELETE).

26. **Single Page Application (SPA) Violation:** RNF-03 and PB-16 explicitly require that CRUD operations not reload the page. However, supply-controller.php forces full reloads using header('Location: ...') redirects after each action.

27. **HTML Error Responses for an API:** If a validation fails in supply, instead of sending a JSON with http_response_code(400) for JavaScript to interpret and report, PHP redirects to error.php?type=supply, completely breaking the front-end integration.

➢ **General and UX**

28. **Inconsistent Spanglish:** The Class Diagram names one of the Roles as Receptionist (Spanish), while the rest of the Diagram, the Backlog, and the source code use the English variant Receptionist.

29. **Opaque Exception Handling:** In both controllers (api_patients.php and supply-controller.php), the catch blocks (Throwable $e) simply wrap errors and return "Server Error" responses or redirects to error.php. The actual error message is neither printed nor logged, making it impossible to debug server failures.

30. **Lack of Logical (Soft-Delete) Functions:** PB-07 and PB-16 request the removal of patients and inventory. The code applies the deleteOne or destroy function, respectively, permanently deleting the records. In clinical and accounting systems, deletions should be logical (changing the status to inactive) to preserve the history of data references and revenue.

❖ **Victoria Díaz**

➢ **Product Backlog Errors**

1. **Technical implementation mixed into User Stories:** Several User Stories related to the Supplies module include technical implementation details instead of focusing only on business value. The backlog references CRUD behavior and operational logic from a technical perspective rather than describing the actor’s needs and expected outcomes. According to Agile practices, User Stories should describe the “what” and the “why,” while implementation details should remain in Acceptance Criteria or technical tasks. This reduces clarity and makes the backlog less understandable for non-technical stakeholders.

➢ **Product Backlog Gaps**

1. **Missing inventory movement functionality:** The backlog states that the system should help administrators avoid running out of supplies and manage inventory efficiently. However, there is no functionality for inventory movements such as stock consumption, stock entries, adjustments, or material usage registration. The system only allows manual CRUD operations over static quantities, which prevents the inventory from reflecting the real operational state of the clinic.
2. **No clinical consumption workflow:** Although the Administrator is responsible for inventory management, the backlog does not include any User Story that allows dentists or assistants to register the consumption of medical supplies during treatments. As a consequence, the inventory only accumulates stock data without representing actual product usage.
3. **Missing supplier and categorization management:** The backlog does not contemplate supplier management, product categories, batch tracking, or inventory classification. These elements are essential in a real inventory system, especially in healthcare environments where traceability and product organization are critical.
4. **Undefined alert mechanisms:** The backlog mentions automatic expiration alerts, but it does not specify how those alerts should behave. There are no definitions for notification channels, alert priorities, recipients, or escalation processes. This creates ambiguity regarding the expected system behavior.
5. **Missing invoicing functionality:** Although the backlog discusses financial management and payment registration, it does not include functionalities for invoice generation, receipts, payment confirmations, or transaction references. These are fundamental components of a complete payment system.
6. **Missing debt and balance management:** The backlog does not define how the system should manage remaining balances, accumulated debt, or partial payments over time. Without these concepts, the financial model remains incomplete.
7. **Missing financial reporting features:** There are no User Stories describing financial reporting, advanced payment searches, historical patient statements, or analytical dashboards. This limits the usefulness of the module for administrative purposes.
8. **Missing relationship with treatments and appointments:** The payment process is not formally connected to treatments, appointments, or invoices within the backlog. As a result, the financial workflow is disconnected from the clinical workflow.

➢ **Product Backlog Inconsistencies**

1. **Inconsistency with non-functional requirements:** The non-functional requirements specify that CRUD operations should work without page reloads. However, the Supplies module relies on traditional HTML forms and server-side redirects, causing full page refreshes after each operation. This directly contradicts the documented system requirements.
2. **Contradiction in payment status logic:** The backlog defines a financial lifecycle based on Pending, Partial, and Completed states. However, the implemented logic directly assigns the Completed status whenever the payment type is "Final," regardless of the actual remaining balance. This contradicts the intended financial behavior.
3. **Inconsistency with asynchronous CRUD operations:** The non-functional requirements state that CRUD operations should occur without page reloads. Nevertheless, the Payments module refreshes the entire page after every operation.

➢ **Class Diagram errors**

1. **Missing UML-defined methods:** The UML class diagram defines an updateStock(qty:int) method inside the Supply entity. However, this method was not implemented in the actual codebase. This creates a direct inconsistency between the designed architecture and the implemented solution.
2. **Missing relationships implementation:** The UML class diagram shows relationships between Payment, Patient, and Receptionist entities. However, the implementation only stores a patientID string without real relational behavior or validation. This means that the conceptual associations represented in the design are not actually enforced by the system.
3. **Missing financial state behavior:** The class diagram suggests that the Payment entity should support proper payment state management. Nevertheless, the implementation only changes status according to payment type instead of using actual financial calculations. This creates a discrepancy between the expected domain behavior and the implemented logic.

➢ **Class Diagram Gaps**

1. **Missing inventory-related entities:** The class diagram is incomplete for a real inventory management system. Critical entities such as Supplier, InventoryMovement, Category, PurchaseOrder, and StockAlert are absent. Without these entities, the model cannot properly support inventory traceability or operational workflows.

2. **Missing financial entities:** The class diagram does not include important financial entities such as Invoice, Receipt, Transaction, or PaymentHistory. These omissions prevent the system from supporting realistic financial processes and limit the extensibility of the payment workflow.
3. **Missing balance management attributes:** The Payment entity lacks attributes such as total debt, remaining balance, invoice references, payment references, and treatment associations. Without these elements, the financial model cannot properly represent real payment scenarios.

➢ **Class Diagram Inconsistencies**

1. **Relationships represented in UML are not implemented:** The UML diagram shows relationships between Administrator and Supply entities. However, the actual implementation does not include ORM relationships, foreign key validation, or user traceability mechanisms. The conceptual design and the code implementation are therefore inconsistent.
2. **Simplified domain model compared to documented requirements:** The Supply entity only stores basic attributes such as name, quantity, expiration date, cost, and status. This simplified model does not align with the complexity implied in the backlog and use cases.
3. **Simplified financial model compared to requirements:** The UML and backlog imply a structured financial workflow capable of handling payment states and patient balances. However, the implemented model only stores isolated payment records with minimal attributes. This creates an inconsistency between the conceptual design and the actual domain model.
4. **Inconsistency between relationships and implementation:** Although the UML visually represents associations between entities, those relationships are not reflected in the actual ORM implementation. The system behaves as if entities were independent rather than interconnected.

➢ **Use Case Diagram Errors**

1. **Incomplete operational workflows:** The use case diagram only represents basic inventory administration actions. Important workflows such as inventory consumption registration, stock adjustment, alert generation, report exporting, and inventory auditing are not

represented. This produces an incomplete representation of the module's real operational needs.

2. **Missing financial workflows:** The use case diagram does not include invoice generation, payment cancellation, refund management, financial history consultation, or receipt generation. These are essential processes in a payment management system and their absence weakens the completeness of the analysis model.
3. **Missing treatment-payment interaction:** The diagram does not represent interactions between treatments and payments. Since clinical services generate financial obligations, this omission disconnects the payment process from the clinic's operational workflow.

➢ **Use Case Diagram Gaps**

1. **Missing actor responsibilities:** The diagram does not specify which actors are responsible for consuming supplies, approving inventory changes, or receiving expiration alerts. This lack of definition creates ambiguity in the system's responsibilities.
2. **Missing reporting and monitoring use cases:** There are no use cases related to reporting, analytics, or inventory monitoring. The absence of these functionalities limits the usefulness of the module for decision-making processes.
3. **Missing notification-related use cases:** The diagram does not include notifications for overdue balances, payment confirmations, or failed transactions. These behaviors are common in financial systems and should be represented.

➢ **Use Case Diagram Inconsistencies**

1. **Difference between documented workflows and implemented behavior:** The use case diagram suggests a more complete inventory process than the one implemented in code. While the diagram implies inventory supervision and operational control, the implemented system only supports simple CRUD operations.
2. **Difference between documented and implemented payment flow:** The use case diagram suggests a more complete payment lifecycle than the one implemented in code. The actual system only performs isolated payment registration without full financial tracking or debt management.

➢ **Code errors**

1. **Missing backend validations:** The Supply.php model does not properly validate duplicated names, invalid characters, maximum field lengths, or unrealistic quantity values. The lack of robust backend validation increases the risk of inconsistent or corrupted data.
2. **Incorrect payment status logic:** The payment status depends exclusively on the payment type:
   ```
   if ($this->paymentType === 'Final') {
      $this->status = 'Completed';
   }
   ```
   This logic ignores remaining balances and accumulated debt, producing incorrect financial states.
3. **Insufficient backend validation:** The backend does not validate patient existence, payment consistency, financial limits, or invalid payment methods. This exposes the system to inconsistent financial records.
4. **Lack of transaction management:** The module does not use transactional operations or rollback mechanisms. If an error occurs during a financial operation, the system may leave inconsistent records.

➢ **Code Gaps**

1. **Missing inventory traceability:** The codebase does not include mechanisms for recording inventory movements, stock adjustments, or historical modifications. There is no audit trail or operational history.
2. **Missing automatic notification system:** Although the system calculates expiration states visually, there is no real notification mechanism such as emails, dashboard alerts, or automated reminders.
3. **Missing financial tracking mechanisms:** The codebase lacks mechanisms for balance tracking, debt accumulation, invoice linkage, or payment history management.
4. **Missing integration with clinical workflow:** Payments are not linked to appointments, treatments, or invoices. This prevents the system from maintaining a coherent operational process.

➢ **Code Inconsistencies**

1. **Inconsistency between architecture definition and implementation:** The project documentation states that the application follows a RESTful MVC architecture with asynchronous CRUD operations. However, the actual implementation behaves as a traditional server-rendered PHP application.

2. **Inconsistency between financial requirements and implementation:** The requirements describe a payment lifecycle with multiple financial states and partial payments. However, the code simplifies payment processing into static status assignment without balance calculations.

➢ **Code Test cases**

1. **TC-SUP-01 — Create a New Supply Successfully:** Verify that an administrator can register a new supply with valid name, quantity, cost, and expiration date information, and confirm that the product appears correctly in the inventory list after saving.
2. **TC-SUP-02 — Create Supply with Empty Name:** Verify that the system prevents the creation of a supply when the name field is left empty and displays an appropriate validation message.
3. **TC-SUP-03 — Register Supply with Expired Date:** Verify that the system rejects a supply registration when the expiration date entered is earlier than the current date.
4. **TC-SUP-04 — Edit Supply Information:** Verify that an existing supply can be updated successfully and that the modified values are reflected correctly in the inventory table.
5. **TC-SUP-05 — Delete Supply Record:** Verify that an administrator can delete an existing supply and confirm that the product no longer appears in the inventory list.
6. **TC-SUP-06 — Verify NextExpiration Status:** Verify that the system automatically classifies a supply as "NextExpiration" when the expiration date is within the next 30 days.
7. **TC-SUP-07 — Verify Expired Status:** Verify that the system automatically classifies a supply as "Expired" when the expiration date has already passed.
8. **TC-SUP-08 — Unauthorized Access to Supplies Module:** Verify that users without administrator permissions cannot access or modify inventory information.
9. **TC-SUP-09 — Search Supply by Name:** Verify that users can search for supplies by entering a product name and that matching results are displayed correctly.
10. **TC-SUP-10 — Register Duplicate Supply:** Verify whether the system prevents the registration of multiple supplies with the same name.
11. **TC-SUP-11 — Register Negative Quantity:** Verify that the system rejects negative quantity values during supply registration.
12. **TC-SUP-12 — Register Negative Cost:** Verify that the system rejects invalid negative cost values when creating or editing a supply.
13. **TC-SUP-13 — Verify Data Persistence After Update:** Verify that updated inventory information remains stored correctly after reloading the page.

14. **TC-SUP-14 — Submit Invalid Characters in Supply Name:** Verify that the system handles invalid characters or malicious input safely during supply creation.
15. **TC-SUP-15 — Verify Inventory Performance with Many Records:** Verify that the inventory module remains functional and responsive when handling a large number of supply records.
16. **TC-PAY-01 — Register Payment Successfully:** Verify that a payment with valid patient information, amount, method, and type can be registered successfully and displayed in the payment history.
17. **TC-PAY-02 — Register Payment Without Patient ID:** Verify that the system prevents payment registration when the patient ID field is empty.
18. **TC-PAY-03 — Register Negative Payment Amount:** Verify that the system rejects payments with negative amount values.
19. **TC-PAY-04 — Register Payment with Future Date:** Verify that the system does not allow payment dates later than the current date.
20. **TC-PAY-05 — Verify Partial Payment Status:** Verify that payments that do not cover the total balance are classified as “Partial”.
21. **TC-PAY-06 — Verify Incorrect Completed Status:** Verify that the system does not classify a payment as “Completed” when the remaining balance has not been fully paid.
22. **TC-PAY-07 — Edit Payment Information:** Verify that an existing payment record can be updated successfully and that the changes are stored correctly.
23. **TC-PAY-08 — Delete Payment Record:** Verify that payment records can be deleted and removed from the payment history.
24. **TC-PAY-09 — Register Payment for Nonexistent Patient:** Verify that the system rejects payments associated with invalid or nonexistent patient IDs.
25. **TC-PAY-10 — Unauthorized Access to Payments Module:** Verify that unauthorized users cannot access or modify payment information.
26. **TC-PAY-11 — Search Payments by Patient ID:** Verify that users can search payment records using a patient identification number.
27. **TC-PAY-12 — Verify Payment History Visualization:** Verify that all registered payments are displayed correctly in the payment history section.
28. **TC-PAY-13 — Register Invalid Payment Method:** Verify that the system rejects unsupported or invalid payment methods.
29. **TC-PAY-14 — Verify Payment Data Persistence:** Verify that edited payment information remains stored correctly after refreshing the module.
30. **TC-PAY-15 — Submit Invalid Characters in Payment Fields:** Verify that the system handles invalid characters or malicious input safely in payment forms.